Quicx

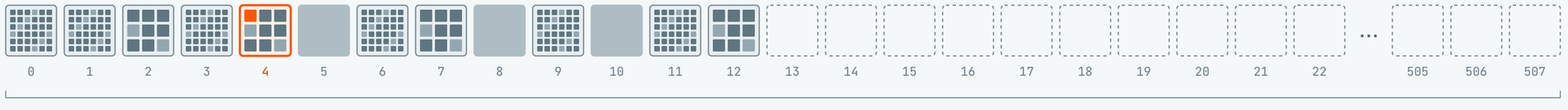
apool v2 · Allocator diagram

adaptive-pool · Nov 2026

Scope: one pool (the task pool, 8 MiB) · quicx/src/apool.c, 341 lines of C

One mapping

[mmap once · 8,339,328 B · never grown, never moved]

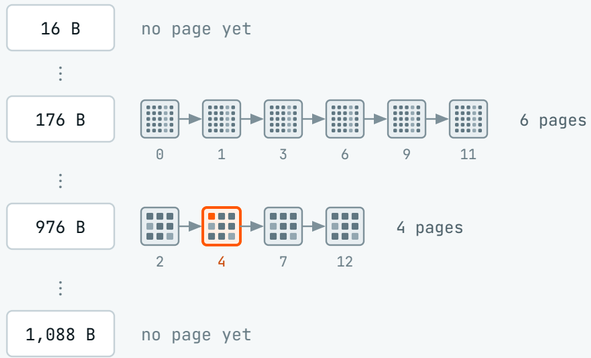


no block headers: a block's page is (address - start) / 16 KiB



Size classes

[68 classes · 16, 32 … 1,088 B]

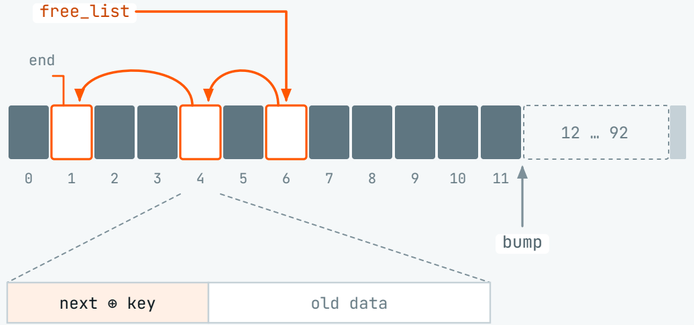


a size owns only the pages it needs right now

slab · Bonwick 1994 · lifetimes · LLAMA 2020 · fullest-first · Hoard (off)

Inside page 3

[176 B blocks · 93 fit · 16 B spare]



freed block 4: its first 8 B point to the next free block

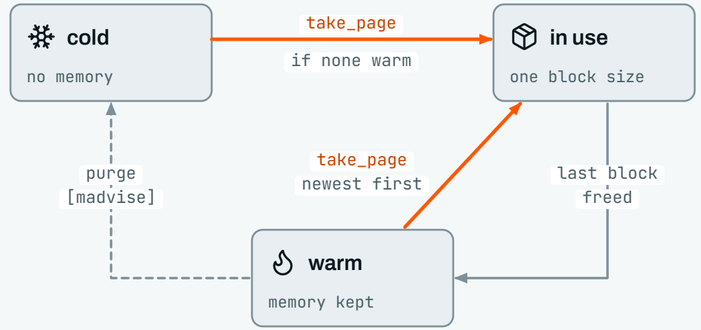
in use · freed · never used

free list in the page · mimalloc 2019

a broken link is caught

Life of a page

[pages change owner; blocks never move]



> 63 warm: the oldest gives memory back

jemalloc 2006

≤ 7 free: only a size with no page may take one (the reserve)

watermarks · Linux

alloc(size)

1. size → class — `c = (size + 15) / 16 - 1` — SIZE
2. a page with room — `else take_page()` — FULL
3. pop the free list — `else bump`
4. hand out the block — `no header · live + 1`

free(ptr)

1. ptr → page — `(ptr - data) / 16 KiB` — INVALID
2. check the block — `owned · on a block edge` — DOUBLE
3. push on the free list — `link XOR-ed`
4. last block? page → warm — `live = 0`

Legend — page in use (finer grid = smaller blocks) · empty, memory kept · memory returned or never used · page 4, followed through · page changes owner · next step / pointer · memory back to the OS · ERR error returned